Predictive Coding Networks

Mathematics, intuition, and a tested PyTorch implementation (with backpropagation side by side)

Assumes you already own backpropagation. The goal is to build the same level of intuition for predictive coding (PC), and to be able to code it, check it against backprop (BP), and know where the two agree and where they do not.

Contents

1 The big picture 1

2 Probabilistic foundations 2

3 Dynamics: inference and learning 4

4 Predictive coding versus backpropagation 5

5 PyTorch implementation 6

6 Verification: what I ran and what came out 9

7 Practical notes 10

8 Formula sheet 11

1 The big picture

1.1 One sentence

Every layer keeps a *belief* x^l about its own activity and uses the layer next to it to *predict* that belief; the only thing that circulates is the *prediction error* $\boldsymbol{\varepsilon}^l = x^l - \mu^l$. Beliefs move to shrink errors (**inference**), then weights move to make the predictions better (**learning**). Both are gradient descent on one scalar, the **energy** F.

Three pictures to keep in your head

- **Video codec.** Do not transmit the frame, transmit frame − prediction. Only surprise costs anything. A PC layer passes up the residual of its prediction, not its raw activity.
- **Springs.** Each ε^l is a spring between the value node x^l and its prediction μ^l. The input (and, in training, the target) are nailed to the wall. Hidden nodes slide until the forces balance. That is inference. Learning is then re-tuning how stiff and how aligned the springs are (the weights) so the settled configuration needs less strain.
- **Explaining away.** If the output is wrong, BP blames weights immediately, with hidden activity held fixed. PC first lets hidden activity *absorb* part of the discrepancy (the network "imagines" the internal state that would have produced the target), and only the *remaining* error trains the weights.

1.2 What PC changes relative to BP

You know BP well, so the contrast in one table:

	Backprop	Predictive coding
Error signal	$\delta^l = \partial\mathcal{L}/\partial a^l$, computed in a separate backward pass	$\varepsilon^l = x^l - \mu^l$, a local quantity that lives in the network itself
Phases	forward, then backward (two different computations)	one dynamical system: relax to equilibrium, then update
Weight update	$\delta^l f(a^{l-1})^\top$ (needs δ^l from far away)	$\varepsilon^l f(x^{l-1})^\top$ (post-synaptic error $\times$ pre-synaptic activity; Hebbian)
What is minimised	loss $\mathcal{L}(\theta)$	$F^\star(\theta) = \min_x F(x, \theta)$
Cost per example	1 fwd + 1 bwd	$\approx T$ (inference steps) $\times$ (fwd-like + bwd-like)
Needs $W^\top$?	yes	yes (top-down error path uses $W^\top$; "weight transport" is only partly removed)
Extra abilities	–	generative modelling, bidirectional inference, arbitrary graphs, local learning

Do not expect PC to be "BP but cheaper"

On a GPU, PC costs more than BP (about T times more). Its appeal is locality, biological plausibility, a probabilistic interpretation, and flexibility, not speed. Section 4 shows exactly when it reproduces BP gradients and when it does not.

1.3 The circuit

Figure 1 shows the local circuit around layer l. There are two kinds of nodes: *value nodes* x^l (circles) and *error nodes* ε^l (squares). Predictions flow up the hierarchy (in the supervised convention used here), errors flow back down.

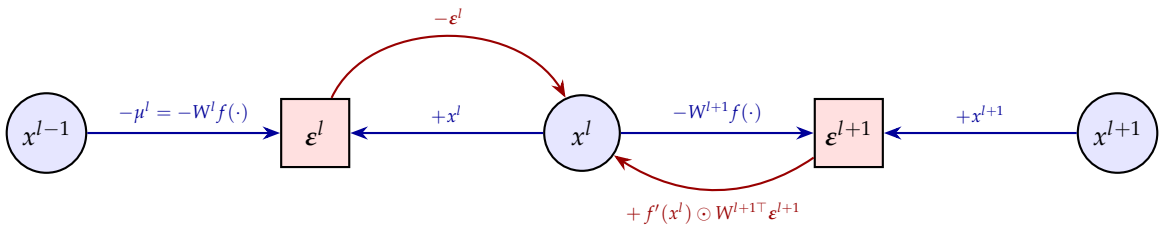


Figure 1: Local circuit. Blue: forward drive that forms errors. Red: the two forces on x^l (own error pulls it toward its prediction; the error above, routed through $W^{l+1\top}$ and f', pushes it toward a value that would better predict x^{l+1}).

2 Probabilistic foundations

2.1 Notation

$l = 0, \ldots, L$	layer index; x^0 is the input, x^L is the output/target
$x^l \in \mathbb{R}^{d_l}$	value nodes ("neural activity", beliefs). Row-batch form in code: shape (B, d_l)
$W^l \in \mathbb{R}^{d_l \times d_{l-1}}$, b^l	weights/bias that map layer $l-1$ to layer l
f, f'	elementwise activation and derivative; $\odot$ is the elementwise product
$\mu^l = W^l f(x^{l-1}) + b^l$	prediction of layer l
$\varepsilon^l = x^l - \mu^l$	prediction error (sign convention: value minus prediction)
$\pi_l > 0$	precision (inverse variance) of layer l
$F(x, \theta)$	energy (variational free energy under a point-mass posterior); $\theta = \{W^l, b^l\}$
γ, α, T	inference step size, learning rate, number of inference steps
a^l, δ^l, $\mathcal{L}$	BP quantities: feed-forward pre-activations, $\partial\mathcal{L}/\partial a^l$, loss

Note that f is applied to x^{l-1} *before* the linear map, so x^l is a pre-activation and the output layer is linear. The input passes through f as well (harmless; use $f = \mathrm{id}$ on x^0 if you prefer).

2.2 The hierarchical Gaussian model

Treat the network as a generative model of the chain $x^0 \to x^1 \to \cdots \to x^L$:

$$p(x^{1:L} \mid x^0; \theta) = \prod_{l=1}^{L} p(x^l \mid x^{l-1}), \qquad p(x^l \mid x^{l-1}) = \mathcal{N}(x^l;\, \mu^l,\, \Sigma_l), \qquad \mu^l = W^l f(x^{l-1}) + b^l. \tag{1}$$

Training pairs (x^0, y) *observe* x^0 and $x^L = y$; the intermediate $x^{1:L-1}$ are latent.

2.3 From log-likelihood to the energy

We want $\ln p(y \mid x^0) = \ln \int p(x^{1:L} \mid x^0)\, dx^{1:L-1}$. For any distribution q over the latents (Jensen),

$$\ln p(y \mid x^0) \;\geq\; \mathbb{E}_q\big[\ln p(x^{1:L} \mid x^0)\big] - \mathbb{E}_q[\ln q] \;=:\; -\mathcal{F}[q]. \tag{2}$$

Choose the crudest variational family, a point mass $q = \delta(x^{1:L-1} - \hat{x})$ (the Laplace/MAP simplification). The entropy term does not depend on $\hat{x}$ or θ and is dropped, leaving

$$F(x, \theta) = -\ln p(x^{1:L} \mid x^0; \theta) + \text{const} = \sum_{l=1}^{L} \left[\tfrac{1}{2}\, \varepsilon^{l\top} \Sigma_l^{-1} \varepsilon^l + \tfrac{1}{2} \ln |\Sigma_l|\right] + \text{const}. \tag{3}$$

With $\Sigma_l = \pi_l^{-1} I$ (fixed, isotropic) the log-determinants are constants and

$$\boxed{F(x, \theta) = \sum_{l=1}^{L} \frac{\pi_l}{2} \left\| x^l - W^l f(x^{l-1}) - b^l \right\|^2} \tag{4}$$

The energy is a sum of squared prediction errors, one per layer, each coupling only neighbouring layers. That locality is the whole reason PC updates are local.

Precision

π_l scales how much layer l's mismatch matters. A large π_L at the output clamps the network hard onto the target; a small π_L is a gentle nudge. In the neuroscience story precision is attention. In Section 4 you will see that π_L is also the dial that interpolates between "PC $\approx$ BP" ($\pi_L \to 0$) and "PC $\neq$ BP" ($\pi_L = 1$). With full covariances use $\Pi_l = \Sigma_l^{-1}$ in place of π_l everywhere below.

2.4 Generative direction (Rao & Ballard) vs supervised direction

The same energy can be arranged two ways. **Supervised / discriminative** (above): input at the bottom, target at the top, predictions flow up, which gives a model of $p(y \mid x)$. **Generative**: data at the bottom, causes at the top, predictions flow *down*, $\mu^l = W^{l+1} f(x^{l+1})$, with a prior on the top layer; this models $p(x)$ and is the classic Rao–Ballard setting.

	Supervised (used for the BP comparison)	Generative (Rao–Ballard)
Prediction	$\mu^l = W^l f(x^{l-1})$	$\mu^l = W^{l+1} f(x^{l+1})$
Clamped	x^0 (input), x^L (target, train only)	x^0 (data)
Energy	$\sum_{l\geq 1} \frac{\pi_l}{2} \|x^l - \mu^l\|^2$	$\sum_{l\geq 0} \frac{\pi_l}{2} \|x^l - \mu^l\|^2$ (top: μ^L = prior mean)
Hidden update	$-\pi_l \varepsilon^l + f'(x^l) \odot W^{l+1\top} \pi_{l+1} \varepsilon^{l+1}$	$-\pi_l \varepsilon^l + f'(x^l) \odot W^{l\top} \pi_{l-1} \varepsilon^{l-1}$
Weight update	$\Delta W^l = \alpha\, \pi_l \varepsilon^l f(x^{l-1})^\top$	$\Delta W^{l+1} = \alpha\, \pi_l \varepsilon^l f(x^{l+1})^\top$

The generative code in Section 5 is the one-latent-layer case: $F = \frac{1}{2}\|x - W f(z)\|^2 + \frac{\lambda}{2}\|z\|^2$.

3 Dynamics: inference and learning

3.1 Inference: gradient flow on the activities

Hold θ fixed and clamp x^0 (and $x^L = y$ in training). The value node x^l ($1 \le l \le L-1$) appears in exactly two terms of (4): in ε^l directly, and inside $f(\cdot)$ in ε^{l+1}. Since $\partial \varepsilon^{l+1}/\partial x^l = -W^{l+1}\,\mathrm{diag}(f'(x^l))$,

$$\boxed{\frac{\partial F}{\partial x^l} = \pi_l\,\varepsilon^l - f'(x^l) \odot \big(W^{l+1\top}\pi_{l+1}\varepsilon^{l+1}\big), \qquad 1 \le l \le L-1} \tag{5}$$

and for an unclamped output $\partial F/\partial x^L = \pi_L \varepsilon^L$. The inference dynamics (Euler, step γ) are

$$x^l \leftarrow x^l - \gamma\,\frac{\partial F}{\partial x^l} = x^l + \gamma\Big(-\pi_l\varepsilon^l + f'(x^l) \odot W^{l+1\top}\pi_{l+1}\varepsilon^{l+1}\Big). \tag{6}$$

Reading the two terms is the best intuition you can build:

- $-\pi_l\varepsilon^l$: "*Be what the layer below predicts.*" A bottom-up pull toward μ^l.
- $+f'(x^l) \odot W^{l+1\top}\pi_{l+1}\varepsilon^{l+1}$: "*Be something that would help predict the layer above.*" The error ε^{l+1} is sent back down through the same weights, gated by the local slope $f'(x^l)$. This term is exactly the BP recursion in disguise (Section 4).

At a fixed point of (6) the two forces cancel, which is the **equilibrium condition**

$$\pi_l\,\varepsilon^l = f'(x^l) \odot W^{l+1\top}\pi_{l+1}\varepsilon^{l+1} \qquad (1 \le l \le L-1). \tag{7}$$

Proposition 1 (Inference is well-posed for linear f)**.** *If $f = \mathrm{id}$, F is a quadratic in the hidden activities with Hessian $H = A^\top \Pi A \succ 0$, where A stacks $\partial\varepsilon^l/\partial x^{1:L-1}$ (identity blocks on the diagonal, $-W^l$ below it, hence full column rank). So the minimiser is unique and gradient descent converges for any $\gamma < 2/\lambda_{\max}(H)$.*

For nonlinear f this holds locally around a minimiser; in practice choose γ small enough that the energy decreases monotonically (Section 7).

3.2 Learning: a Hebbian rule

With x at (or near) equilibrium, differentiate (4) with respect to the parameters. Since $\varepsilon^l = x^l - W^l f(x^{l-1}) - b^l$,

$$\boxed{\frac{\partial F}{\partial W^l} = -\pi_l\,\varepsilon^l\,f(x^{l-1})^\top, \qquad \frac{\partial F}{\partial b^l} = -\pi_l\,\varepsilon^l} \qquad \Rightarrow \qquad \Delta W^l = \alpha\,\pi_l\,\varepsilon^l f(x^{l-1})^\top. \tag{8}$$

This is "post-synaptic error $\times$ pre-synaptic activity": it needs only the two neurons the synapse connects. No global backward pass is needed to *apply* it, in contrast with $\delta^l f(a^{l-1})^\top$ in BP where δ^l travels from the output.

3.3 The EM view and the algorithm

Because $F = -\ln p + \text{const}$, PC is (hard) variational EM: the **E-step** infers the latent activities $x^\star(\theta) = \arg\min_x F(x,\theta)$; the **M-step** takes a gradient step on θ with x fixed at $x^\star$. In practice you do not run the E-step to full convergence for each example, you run T steps and update.

Algorithm: one PC training step (supervised)

1. Clamp $x^0 \leftarrow$ input. Initialise hidden $x^l \leftarrow a^l$ by a feed-forward pass ($x^l = \mu^l(x^{l-1})$, so all hidden errors start at 0).
2. Clamp $x^L \leftarrow y$. Now the only non-zero error is $\varepsilon^L = y - a^L$.
3. **Inference:** repeat T times: compute all $\varepsilon^l = x^l - \mu^l$, then update every hidden x^l with (6) (synchronously).
4. **Learning:** with the final x and ε, update $W^l \leftarrow W^l + \alpha\pi_l\varepsilon^l f(x^{l-1})^\top$, $b^l \leftarrow b^l + \alpha\pi_l\varepsilon^l$ (or feed $-\partial F/\partial\theta$ to any optimizer).
5. **Test time:** feed-forward a^L as the prediction (or run inference with x^L free).

3.4 Worked example: the two-layer linear network, in closed form

Take $f = \mathrm{id}$, $b = 0$, $\pi = 1$, dims $d_0 \to d_1 \to d_2$, and write $h = W^1 x^0$ and $r = y - W^2 W^1 x^0$ (the BP residual). Then

$$F(x^1) = \tfrac{1}{2}\|x^1 - h\|^2 + \tfrac{1}{2}\|y - W^2 x^1\|^2.$$

Put $u = x^1 - h$; then $y - W^2 x^1 = r - W^2 u$, and $\partial F/\partial u = u - W^{2\top}(r - W^2 u) = 0$ gives

$$u^\star = (I + W^{2\top} W^2)^{-1} W^{2\top} r, \qquad \varepsilon^2 = r - W^2 u^\star = (I + W^2 W^{2\top})^{-1} r, \qquad \varepsilon^1 = u^\star = W^{2\top} \varepsilon^2. \tag{9}$$

(The middle equality is the Woodbury identity.) Substituting back,

$$\boxed{F^\star = \tfrac{1}{2}\, r^\top (I + W^2 W^{2\top})^{-1}\, r \;\le\; \tfrac{1}{2}\|r\|^2 = \mathcal{L}} \tag{10}$$

Compare the weight updates. PC: $\Delta W^1 = \alpha\, \varepsilon^1 x^{0\top} = \alpha\, W^{2\top}(I + W^2 W^{2\top})^{-1} r\, x^{0\top}$. BP: $\Delta W^1 = \alpha\, W^{2\top} r\, x^{0\top}$. So **PC is BP with the output residual pre-conditioned by** $(I + W^2 W^{2\top})^{-1}$: part of r has been explained away by moving x^1, and the larger $\|W^2\|$ is, the more is explained away. The error that trains the weights is never larger than the BP error ($\|\varepsilon^2\| \le \|r\|$), which acts like an implicit trust region. I verified (10) numerically (Section 6).

4 Predictive coding versus backpropagation

4.1 BP in the same notation

Feed-forward values: $a^0 = x^0$, $a^l = W^l f(a^{l-1}) + b^l$. Loss $\mathcal{L} = \tfrac{1}{2}\|a^L - y\|^2$. Define $\delta^l = \partial \mathcal{L}/\partial a^l$:

$$\delta^L = a^L - y, \qquad \delta^l = f'(a^l) \odot W^{l+1\top} \delta^{l+1}, \qquad \frac{\partial \mathcal{L}}{\partial W^l} = \delta^l f(a^{l-1})^\top, \quad \frac{\partial \mathcal{L}}{\partial b^l} = \delta^l. \tag{11}$$

4.2 The structural resemblance

Put $\pi_l = 1$ and compare (7) with (11). The equilibrium errors satisfy $\varepsilon^l = f'(x^l) \odot W^{l+1\top} \varepsilon^{l+1}$, the *same backward recursion* as δ^l, and the PC gradient $\partial F/\partial W^l = -\varepsilon^l f(x^{l-1})^\top$ matches $\partial \mathcal{L}/\partial W^l = \delta^l f(a^{l-1})^\top$ if $\varepsilon^l = -\delta^l$. So BP is hiding inside the equilibrium of PC. Three things can break the match: (i) the terminal condition (ε^L versus $-\delta^L = y - a^L$); (ii) f', f evaluated at x instead of a; (iii) the sign and scale. The next two results say when they hold.

Theorem 1 (Fixed-prediction assumption: exact BP)**.** *Freeze the predictions at their feed-forward values, $\mu^l \equiv a^l$ (so $\varepsilon^l = x^l - a^l$), evaluate Jacobians at a, clamp $x^L = y$, and initialise $x^l = a^l$ for $l < L$. The inference dynamics*

$$x^l \leftarrow x^l + \gamma\big(-\varepsilon^l + f'(a^l) \odot W^{l+1\top} \varepsilon^{l+1}\big), \qquad 1 \le l \le L-1,$$

converge to $\varepsilon^l = -\delta^l$ for every $l = 1, \dots, L$. For $\gamma = 1$ the errors are exact after $L - 1$ synchronous steps.

Proof. At the fixed point, $\varepsilon^L = y - a^L = -\delta^L$ (by clamping and frozen prediction) and $\varepsilon^l = f'(a^l) \odot W^{l+1\top} \varepsilon^{l+1}$, which is the BP recursion (11) with sign flipped; induct downward from L. For $\gamma = 1$, $x^l_{t+1} - a^l = f'(a^l) \odot W^{l+1\top} \varepsilon^{l+1}_t$, i.e. $\varepsilon^l_{t+1} = f'(a^l) \odot W^{l+1\top} \varepsilon^{l+1}_t$ with $\varepsilon^l_0 = 0$ ($l < L$): the exact error reaches layer $L - t$ at step t. Hence inference is a backward sweep *unrolled in time*. □

This is the cleanest statement of "PC implements BP" (Millidge et al.; Song et al. 2020). Its weakness: freezing the predictions is not what the real dynamics do, since $\mu^L = W^L f(x^{L-1})$ changes as soon as x^{L-1} moves.

Theorem 2 (Weak output precision: BP as a limit of the true dynamics)**.** *Let $\pi_l = 1$ for $l < L$ and $\pi_L = \pi$. Let $g_{PC} = \partial F/\partial W^l$ at equilibrium of the* full *dynamics* (6) *(no frozen predictions). Then*

$$g_{PC} = \pi \frac{\partial \mathcal{L}}{\partial W^l} + O(\pi^2) \qquad (\pi \to 0),$$

and likewise for the biases. So $g_{PC}/\pi \to \nabla \mathcal{L}$ linearly in π.

Proof sketch. At $\pi = 0$ the output term vanishes; the energy $\sum_{l<L} \frac{1}{2}\|\varepsilon^l\|^2$ has the minimiser $x = a$ (all errors 0) with positive definite Hessian (identity blocks, as in Proposition 1). By the implicit function theorem the equilibrium $x^\star(\pi)$ is smooth with $x^\star(0) = a$, so $x^{\star l} - a^l = O(\pi)$. Define $\tilde{\varepsilon}^l = \pi_l \varepsilon^l$ (the precision-weighted error). Then (7) reads $\tilde{\varepsilon}^l = f'(x^l) \odot W^{l+1\top} \tilde{\varepsilon}^{l+1}$, and $\tilde{\varepsilon}^L = \pi\,(y - \mu^L) = \pi(y - a^L) + O(\pi^2)$ because $\mu^L = a^L + O(\pi)$. Backward induction gives $\tilde{\varepsilon}^l = -\pi\delta^l + O(\pi^2)$ (and $f(x^{l-1}) = f(a^{l-1}) + O(\pi)$). Finally $\partial F/\partial W^l = -\tilde{\varepsilon}^l f(x^{l-1})^\top = \pi\,\delta^l f(a^{l-1})^\top + O(\pi^2)$. □

Theorem 2 needs no frozen predictions. Its price: the signal is scaled by π, so you must raise the learning rate by $1/\pi$ (automatic with Adam, which is scale-invariant).

4.3 The general case: PC descends a different objective

For $\pi_L = 1$ the match is only approximate. The exact statement is an envelope-theorem fact.

Proposition 2 (PC learning is exact gradient descent on $F^\star$)**.** *Let $F^\star(\theta) = \min_x F(x, \theta) = F(x^\star(\theta), \theta)$ with $x^\star$ a differentiable (non-degenerate) minimiser. Then*

$$\nabla_\theta F^\star(\theta) = \partial_\theta F(x, \theta)\big|_{x = x^\star(\theta)},$$

i.e. the Hebbian update (8) evaluated at equilibrium is exactly $-\alpha \nabla_\theta F^\star$.

Proof. $\nabla_\theta F^\star = \partial_\theta F + \left(\partial x^\star/\partial\theta\right)^\top \partial_x F$ and $\partial_x F(x^\star, \theta) = 0$. □

Since $x^\star$ minimises over x and $x = a$ (with $x^L = y$) is one candidate, $F^\star(\theta) \le F(a, \theta) = \mathcal{L}(\theta)$ for $\pi_L = 1$. For the linear example (10), $F^\star$ is the loss measured through the metric $(I + W^2 W^{2\top})^{-1}$. So **PC with $\pi_L = 1$ is not BP**: it optimises a "relaxed" loss. They share the global optima in the linear example (both vanish iff $r = 0$), but trajectories, and in general stationary points, can differ. Related work interprets this as an implicit-SGD-like or trust-region-like update, and as "prospective configuration" (settle activity to what it *should* be after learning, then consolidate weights); see references.

What to remember

(1) Inference with frozen predictions is a time-unrolled backward pass and gives exact BP gradients. (2) With full dynamics and small output precision $\pi_L \to 0$, the PC gradient is $\pi_L \times$ the BP gradient. (3) With $\pi_L = 1$, PC is gradient descent on $F^\star(\theta) = \min_x F$, a damped, preconditioned cousin of the loss. (4) In all cases the weight update is local; the cost is the T inference steps.

5 PyTorch implementation

Everything below is the code I ran (PyTorch 2.14, float64 for the checks). Layers are stored in `nn.Linear`: `net.fc[l-1]` is (W^l, b^l), so `weight` has shape (d_l, d_{l-1}) and `x[l]` has shape (B, d_l), which means $W^{l+1\top}\varepsilon^{l+1}$ becomes `eps[l+1] @ W`.

Math	Code
$\mu^l = W^l f(x^{l-1}) + b^l$	`self.fc[l-1](self.act.f(x[l-1]))`
$\varepsilon^l = x^l - \mu^l$	`x[l] - self.mu(l, x[l-1])`
$\pi_l\varepsilon^l - f'(x^l) \odot W^{l+1\top}\pi_{l+1}\varepsilon^{l+1}$	`g = pi[l]*eps[l]; g -= df(x[l]) * ((pi[l+1]*eps[l+1]) @ W_up)`
$\partial F/\partial W^l = -\pi_l\varepsilon^l f(x^{l-1})^\top$	`-(e.T @ f(x[l-1])) / B` (batch mean)

5.1 The predictive coding network

Design choice: `set_grads` writes $\partial F/\partial\theta$ into `.grad`, so any PyTorch optimizer (SGD, Adam) works unchanged.

```
import torch
import torch.nn as nn


class Act:
```

```
    """Activation f and its derivative f'."""
    def __init__(self, f, df):
        self.f, self.df = f, df


tanh = Act(torch.tanh, lambda a: 1 - torch.tanh(a) ** 2)
relu = Act(torch.relu, lambda a: (a > 0).to(a.dtype))


class PCN(nn.Module):
    """
    Supervised predictive coding network.
    Layers l = 0..L.  x^0 = input (clamped), x^L = target (clamped in training).
    Prediction:  mu^l = W^l f(x^{l-1}) + b^l   (= self.fc[l-1])
    Error:       eps^l = x^l - mu^l
    Energy:      F = sum_l (pi_l / 2) ||eps^l||^2
    """
    def __init__(self, dims, act=tanh, pi=None):
        super().__init__()
        self.L, self.act = len(dims) - 1, act
        self.fc = nn.ModuleList(nn.Linear(dims[i], dims[i + 1]) for i in range(self.L))
        self.pi = pi if pi is not None else [1.0] * (self.L + 1)  # pi[l], l = 1..L

    def mu(self, l, x_prev):                        # mu^l
        return self.fc[l - 1](self.act.f(x_prev))

    @torch.no_grad()
    def feedforward(self, x0):                      # a^0..a^L  (the "BP forward pass")
        a = [x0]
        for l in range(1, self.L + 1):
            a.append(self.mu(l, a[-1]))
        return a

    def errors(self, x):                            # eps^1..eps^L  (eps[0] unused)
        return [None] + [x[l] - self.mu(l, x[l - 1]) for l in range(1, self.L + 1)]

    def energy(self, x):                            # per-sample energy, shape (B,)
        eps = self.errors(x)
        return sum(0.5 * self.pi[l] * (eps[l] ** 2).sum(1) for l in range(1, self.L + 1))

    @torch.no_grad()
    def infer(self, x0, y=None, T=100, gamma=0.1):
        """Inference: gradient descent on F w.r.t. the hidden activities."""
        x = self.feedforward(x0)                    # init at feedforward pass
        if y is not None:
            x[self.L] = y.clone()                   # clamp output to the target
        free = range(1, self.L) if y is not None else range(1, self.L + 1)
        for _ in range(T):
            eps = self.errors(x)
            dx = {}
            for l in free:
                g = self.pi[l] * eps[l]                                  # own error
                if l < self.L:                                           # error from above
                    W_up = self.fc[l].weight                             # W^{l+1}
                    g = g - self.act.df(x[l]) * ((self.pi[l + 1] * eps[l + 1]) @ W_up)
                dx[l] = -gamma * g
            for l in free:                                               # synchronous update
                x[l] = x[l] + dx[l]
        return x

    @torch.no_grad()
    def set_grads(self, x):
        """Write dF/dW, dF/db (batch mean) into .grad  ->  any torch optimizer works."""
        eps, B = self.errors(x), x[0].shape[0]
        for l in range(1, self.L + 1):
            e = self.pi[l] * eps[l]
            self.fc[l - 1].weight.grad = -(e.T @ self.act.f(x[l - 1])) / B   # -eps f(x)^T
            self.fc[l - 1].bias.grad = -e.mean(0)

    def train_step(self, opt, x0, y, T=50, gamma=0.1):
        opt.zero_grad()
        x = self.infer(x0, y, T, gamma)             # E-step
        self.set_grads(x)                           # M-step
        opt.step()
        return self.energy(x).mean().item()
```

5.2 Backpropagation, manual and autograd

`backprop_manual` is equation (11) line by line (it also returns δ^l so you can compare it with $-\varepsilon^l$). `backprop_autograd` is the standard PyTorch route and fills `.grad` just like `set_grads`, so BP and PC can share one training loop.

```python
# ---------------------------------------------------------------------------
# Backpropagation in the same notation
# ---------------------------------------------------------------------------
@torch.no_grad()
def backprop_manual(net, x0, y):
    """Manual BP.  delta^l = dLoss/da^l,  Loss = 1/2 ||a^L - y||^2 (batch mean)."""
    a, B, L = net.feedforward(x0), x0.shape[0], net.L
    delta = [None] * (L + 1)
    delta[L] = a[L] - y
    for l in range(L - 1, 0, -1):
        delta[l] = net.act.df(a[l]) * (delta[l + 1] @ net.fc[l].weight)    # W^{l+1}
    gW = [None] + [delta[l].T @ net.act.f(a[l - 1]) / B for l in range(1, L + 1)]
    gb = [None] + [delta[l].mean(0) for l in range(1, L + 1)]
    loss = 0.5 * ((a[L] - y) ** 2).sum(1).mean().item()
    return loss, gW, gb, delta


def backprop_autograd(net, x0, y):
    net.zero_grad()
    a = x0
    for l in range(1, net.L + 1):                 # forward pass WITH graph
        a = net.mu(l, a)
    loss = 0.5 * ((a - y) ** 2).sum(1).mean()
    loss.backward()
    gW = [None] + [net.fc[l - 1].weight.grad.clone() for l in range(1, net.L + 1)]
    gb = [None] + [net.fc[l - 1].bias.grad.clone() for l in range(1, net.L + 1)]
    return loss.item(), gW, gb
```

5.3 The two BP↔PC bridges

The first is Theorem 1 (frozen predictions). The second computes inference with autograd on the energy (only adjacent layers appear in each term, so this is still local and the graph is shallow). It is the quickest way to validate (5) by hand and to try new energies.

```python
# ---------------------------------------------------------------------------
# Fixed-prediction assumption (FPA): mu frozen at feedforward values
# ---------------------------------------------------------------------------
@torch.no_grad()
def infer_fpa(net, x0, y, T=200, gamma=0.5):
    a = net.feedforward(x0)
    x = [t.clone() for t in a]
    x[net.L] = y.clone()
    for _ in range(T):
        eps = [None] + [x[l] - a[l] for l in range(1, net.L + 1)]       # mu^l = a^l frozen
        dx = {}
        for l in range(1, net.L):
            g = eps[l] - net.act.df(a[l]) * (eps[l + 1] @ net.fc[l].weight)
            dx[l] = -gamma * g
        for l in range(1, net.L):
            x[l] = x[l] + dx[l]
    return x, [None] + [x[l] - a[l] for l in range(1, net.L + 1)]


# ---------------------------------------------------------------------------
# Autograd version of PC inference (checks the hand-derived formulas)
# ---------------------------------------------------------------------------
def infer_autograd(net, x0, y, T=100, gamma=0.1):
    with torch.no_grad():
        x = net.feedforward(x0)
        x[net.L] = y.clone()
    for l in range(1, net.L):
        x[l].requires_grad_()
    for _ in range(T):
        F = net.energy(x).sum()
        grads = torch.autograd.grad(F, x[1:net.L])
        with torch.no_grad():
```

```
            for l, g in zip(range(1, net.L), grads):
                x[l] -= gamma * g
    return [t.detach() for t in x]
```

5.4 Training loop

```
net = PCN([784, 256, 256, 10], tanh, pi=[1, 1, 1, 0.3])   # pi[l] for l=1..L
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for epoch in range(n_epochs):
    for xb, yb in loader:                # xb: (B, 784), yb: one-hot (B, 10)
        F = net.train_step(opt, xb, yb, T=40, gamma=0.2)
    pred = net.feedforward(x_test)[-1].argmax(1)       # test: feed-forward sweep
    acc = (pred == y_test).float().mean()
```

For the BP baseline replace `train_step` by `opt.zero_grad()`, `backprop_autograd(net, xb, yb)`, `opt.step()`.

5.5 Generative predictive coding (Rao–Ballard, one latent layer)

Energy $F = \frac{1}{2}\|x - Wf(z)\|^2 + \frac{\lambda}{2}\|z\|^2$. Inference: $\dot{z} = f'(z) \odot W^\top \varepsilon - \lambda z$. Learning: $\Delta W = \alpha\, \varepsilon f(z)^\top$.

```
# ------------------------------------------------------------------------------
# Generative (Rao & Ballard style) predictive coding: z -> x
# ------------------------------------------------------------------------------
class GenPC:
    """F = 1/2 ||x - W f(z)||^2 + (lam/2) ||z||^2   (x observed, z latent)."""
    def __init__(self, d_obs, d_lat, act=tanh, lam=1e-2, seed=0, dtype=torch.float64):
        g = torch.Generator().manual_seed(seed)
        self.W = torch.randn(d_obs, d_lat, generator=g, dtype=dtype) / d_lat ** 0.5
        self.act, self.lam = act, lam

    def infer(self, X, T=100, gamma=0.1):
        z = torch.zeros(X.shape[0], self.W.shape[1], dtype=X.dtype)
        for _ in range(T):
            eps = X - self.act.f(z) @ self.W.T                          # eps = x - mu
            z = z + gamma * (self.act.df(z) * (eps @ self.W) - self.lam * z)
        return z, X - self.act.f(z) @ self.W.T

    def step(self, X, lr=0.05, T=100, gamma=0.1):
        z, eps = self.infer(X, T, gamma)
        self.W += lr * eps.T @ self.act.f(z) / X.shape[0]               # Hebbian: eps * f(z)
        return 0.5 * (eps ** 2).sum(1).mean().item()
```

6 Verification: what I ran and what came out

The script `check.py` climbs a debugging ladder. Each rung validates one piece of the math against an independent computation (net $5 \to 8 \to 7 \to 3$, tanh, float64, random data).

Check	Result
Manual BP (11) vs `loss.backward()`	$\max \lvert\Delta\rvert = 1.4 \times 10^{-17}$
Hand-derived $\partial F/\partial x$ (5) vs autograd (random x, non-unit π_l)	5.6×10^{-17}
Hand-derived $\partial F/\partial W$ (8) vs autograd	0 (identical)
Manual inference vs autograd inference (300 steps)	1.1×10^{-16}
Theorem 1: $\max \lvert\varepsilon^l + \delta^l\rvert$ (converged)	1.7×10^{-16}
Theorem 1: same with $\gamma = 1$, only $L - 1 = 2$ steps	1.1×10^{-16}
Linear net: $F^\star$ from inference vs closed form (10)	0.452175… both; $\mathcal{L} = 0.6314 \geq F^\star$
Linear net: ε^2 vs $(I + W^2 W^{2\top})^{-1} r$	1.1×10^{-16}

Weak-nudge limit (Theorem 2). Relative error $\|g_{PC}/\pi_L - g_{BP}\|/\|g_{BP}\|$ and cosine similarity:

π_L	1	0.3	0.1	0.03	0.01	0.003	0.001
cosine	0.406	0.897	0.989	0.9990	0.99989	0.99999	0.999999
rel. error	0.97	0.47	0.18	0.058	0.020	0.0059	0.0020

The error falls linearly with π_L, as the $O(\pi^2)/\pi$ remainder predicts. At $\pi_L = 1$ the PC gradient points in a quite different direction from the BP gradient (cosine 0.41 for this random network), which is the preconditioning of Section 4.

Toy training run (3-class spirals, 480 train / 120 test, MLP 2-64-64-3, tanh, MSE-style one-hot targets, Adam 3×10^{-3}, 150 epochs, batch 64; PC: $T = 40$, $\gamma = 0.2$; test-time prediction by feed-forward pass; one seed):

	train acc.	test acc.
BP	1.000	0.992
PC, $\pi_L = 1$	0.875	0.850
PC, $\pi_L = 0.3$	0.994	0.992
PC, $\pi_L = 0.1$	0.985	0.983

Generative PC (`GenPC`, 10-d data from a 3-d latent): reconstruction energy 3.67 → 0.38 (iteration 100) → 0.31 (iteration 600), versus 5.94 for the data energy itself.

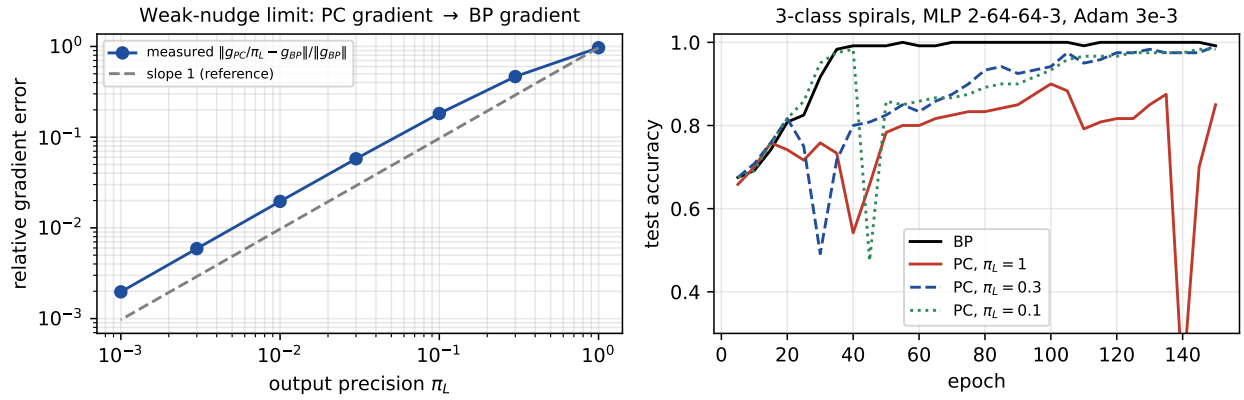


Figure 2: Left: relative error between g_{PC}/π_L and the BP gradient shrinks linearly with π_L. Right: test accuracy on the toy task.

How far to trust this

The gradient checks are exact and general. The training run is a single seed on a tiny problem, with minimal tuning for PC (I tried Adam learning rates 10^{-2} and 3×10^{-2} at $\pi_L = 1$, which were worse: test accuracy 0.72 and 0.49). That $\pi_L = 1$ lagged while $\pi_L = 0.3$ matched BP is consistent with the preconditioning/damping analysis, but I did not test that explanation, and it should not be read as a general ranking of PC and BP.

7 Practical notes

- **Initialise at the feed-forward pass.** Then hidden errors start at 0 and inference begins at the BP-like solution; random initialisation wastes steps.
- **Errors travel one layer per step**, so $T \gtrsim L$ at the very least; $T = 20$–100 is typical. Deeper networks need more.
- **Step size γ.** Stability needs $\gamma < 2/\lambda_{\max}(H)$, with $H \approx I + J^\top J$ and $J = Wf'$. Start at 0.1–0.2; if the energy rises, halve γ.
- **Monitor the energy trace within inference.** F_t should decrease monotonically. It is the best single debugging signal.
- **Output precision π_L is the main knob** (Section 4); I would sweep it first.

- **Use a standard optimizer.** Assign `p.grad` from $\partial F/\partial\theta$ and call `opt.step()`.
- **Prediction.** Feed-forward a^L (cheap), or run inference with x^L free and read x^L (costs T steps; the more "PC" answer).
- **Cost.** Roughly T forward-like plus T backward-like matrix products per step (versus 1 and 1). Layers can update in parallel within a step, which is the hardware-friendly aspect.
- **Weight symmetry.** The top-down path uses $W^\top$. Using a separate learned feedback matrix is a known relaxation, but it is a different algorithm, so test it separately.

Debugging ladder (do these in order):

1. autograd $\partial F/\partial x$ and $\partial F/\partial W$ match your manual formulas;
2. FPA inference with $\gamma = 1$ gives $\varepsilon^l = -\delta^l$ after $L-1$ steps;
3. as $\pi_L \to 0$, $g_{PC}/\pi_L \to g_{BP}$;
4. linear network matches the closed form (10);
5. only then train.

Common confusions. (a) The PC output error ε^L equals the loss gradient only under frozen predictions or $\pi_L \to 0$. (b) Sign conventions differ across papers ($\varepsilon = x - \mu$ here); check before comparing formulas. (c) The energy F is not the training loss: for $\pi_L = 1$, $F^\star \le \mathcal{L}$. (d) "Local" refers to the weight update and the error computation; inference still needs many iterations.

8 Formula sheet

Prediction / error	$\mu^l = W^l f(x^{l-1}) + b^l, \quad \varepsilon^l = x^l - \mu^l$
Energy	$F = \sum_{l=1}^{L} \frac{\pi_l}{2} \lVert\varepsilon^l\rVert^2$
Hidden gradient	$\dfrac{\partial F}{\partial x^l} = \pi_l \varepsilon^l - f'(x^l) \odot W^{l+1\top} \pi_{l+1} \varepsilon^{l+1}$
Inference	$x^l \leftarrow x^l - \gamma\, \partial F/\partial x^l \quad (l = 1..L-1;\ x^0, x^L \text{ clamped})$
Equilibrium	$\pi_l \varepsilon^l = f'(x^l) \odot W^{l+1\top} \pi_{l+1} \varepsilon^{l+1}$
Learning	$\Delta W^l = \alpha \pi_l \varepsilon^l f(x^{l-1})^\top, \quad \Delta b^l = \alpha \pi_l \varepsilon^l$
BP	$\delta^L = a^L - y,\ \ \delta^l = f'(a^l) \odot W^{l+1\top} \delta^{l+1},\ \ \nabla_{W^l}\mathcal{L} = \delta^l f(a^{l-1})^\top$
Exact BP (frozen μ)	$\varepsilon^l = -\delta^l$; with $\gamma = 1$ exact after $L-1$ steps
Weak nudge	$\nabla_{W^l} F = \pi_L \nabla_{W^l}\mathcal{L} + O(\pi_L^2)$
Envelope	$\nabla_\theta F^\star = \partial_\theta F|_{x^\star}, \quad F^\star \le \mathcal{L}$ (for $\pi_L = 1$)
Linear 2-layer	$\varepsilon^2 = (I + W^2 W^{2\top})^{-1} r,\ \ F^\star = \frac{1}{2} r^\top (I + W^2 W^{2\top})^{-1} r$

References and further reading

- Rao & Ballard (1999). Predictive coding in the visual cortex. *Nature Neuroscience*.
- Friston (2005). A theory of cortical responses. *Phil. Trans. R. Soc. B*.
- Bogacz (2017). A tutorial on the free-energy framework for modelling perception and learning. *J. Math. Psychology*. (Best gentle derivation.)
- Whittington & Bogacz (2017). An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*.
- Millidge, Tschantz & Buckley (2022). Predictive coding approximates backprop along arbitrary computation graphs. *Neural Computation*.
- Song, Lukasiewicz, Xu & Bogacz (2020). Can the brain do backpropagation? Exact implementation of backpropagation in predictive coding networks. *NeurIPS*.
- Song et al. (2024). Inferring neural activity before plasticity as a foundation for learning beyond backpropagation. *Nature Neuroscience*. (Prospective configuration.)

I wrote these citations from memory without looking them up; verify titles, venues and years before citing them in your own work. The theorems above are derived and numerically checked in this document, not taken from those papers.